Creating a Machine Learning Model for Predicting Diabetes

David Pan

PSTAT 131

Contents

Overview **1**

Exploratory Data Analysis **2**

Correlation matrix 9

Training and Fitting Models **10**

Model Setup 10

Create workflows 11

Create grids 12

Tune parameters 13

Fit models without tuned parameters 19

Compare area under ROC curve 19

Model Selection **21**

Assessing Performance 21

Confusion Matrix 22

ROC curve 22

Conclusion **23**

Overview

Source: https://www.kaggle.com/datasets/iammustafatz/diabetes-prediction-dataset

This dataset contains medical and demographic information sourced from patients, along with their diabetes status (positive/negative). The dataset encompasses attributes such as age, gender, body mass index (BMI), presence of hypertension, heart disease, smoking background, HbA1c levels, and blood glucose readings. This data is valuable to healthcare experts in pinpointing individuals potentially susceptible to diabetes and coming up with possible treatments.

The primary research question of interest is if it is possible to accurately predict whether or not a patient will have diabetes based on other health factors. If so, it would also be important to know which variables in particular are useful in coming up with this prediction.

I will be using a classification approach because the variable being predicted ('diabetes') only has 2 possible outcomes: 0 or 1. The plan is to fit multiple different models to the data and select the one with the highest predicted accuracy, or area under the ROC curve. I will be using logistic regression, k-nearest neighbors, linear discriminant analysis, quadratic discriminant analysis, elastic net regression, random forest, boosted trees, and support vector machine models to fit the data. The ultimate goal of this project is to construct a predictive model that can predict 'diabetes' with a consistently high accuracy.

Exploratory Data Analysis

```r
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.2     v readr     2.1.4
## v forcats   1.0.0     v stringr   1.5.0
## v ggplot2   3.4.2     v tibble    3.2.1
## v lubridate 1.9.2     v tidyr     1.3.0
## v purrr     1.0.1
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(tidymodels)
```

```
## -- Attaching packages -------------------------------------- tidymodels 1.1.0 --
## v broom        1.0.4     v rsample      1.1.1
## v dials        1.2.0     v tune         1.1.1
## v infer        1.0.4     v workflows    1.1.3
## v modeldata    1.2.0     v workflowsets 1.0.1
## v parsnip      1.1.0     v yardstick    1.2.0
## v recipes      1.0.7
## -- Conflicts ----------------------------------------- tidymodels_conflicts() --
## x scales::discard() masks purrr::discard()
## x dplyr::filter()   masks stats::filter()
## x recipes::fixed()  masks stringr::fixed()
## x dplyr::lag()      masks stats::lag()
## x yardstick::spec() masks readr::spec()
## x recipes::step()   masks stats::step()
## * Dig deeper into tidy modeling with R at https://www.tmwr.org
```

```r
library(tidyselect)
library(ggplot2)
library(corrplot)
```

```
## corrplot 0.92 loaded
```

```r
library(discrim)
```

```
##
## Attaching package: 'discrim'
##
## The following object is masked from 'package:dials':
##
##     smoothness
```

```r
library(kknn)
library(glmnet)
```

```
## Loading required package: Matrix
##
## Attaching package: 'Matrix'
##
## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack
##
## Loaded glmnet 4.1-7
```

```r
library(yardstick)
library(themis)
```

The first step of this project is to load in the data and become familiar with it. It looks like the data is already tidy, which is convenient because it means I won't have to melt or pivot any columns.

```r
diabetes_data <- read_csv('diabetes_prediction_dataset.csv')
```

```
## Rows: 100000 Columns: 9
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (2): gender, smoking_history
## dbl (7): age, hypertension, heart_disease, bmi, HbA1c_level, blood_glucose_l...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
head(diabetes_data)
```

```
## # A tibble: 6 x 9
##   gender   age hypertension heart_disease smoking_history   bmi HbA1c_level
##   <chr>  <dbl>        <dbl>         <dbl> <chr>           <dbl>       <dbl>
## 1 Female    80            0             1 never            25.2         6.6
## 2 Female    54            0             0 No Info          27.3         6.6
## 3 Male      28            0             0 never            27.3         5.7
## 4 Female    36            0             0 current          23.4         5
## 5 Male      76            1             1 current          20.1         4.8
## 6 Female    20            0             0 never            27.3         6.6
## # i 2 more variables: blood_glucose_level <dbl>, diabetes <dbl>
```

```r
dim(diabetes_data)
```

```
## [1] 100000      9
```

The dataset has 100,000 observations and 9 variables. We are trying to predict diabetes, so there will be a total of 8 predictors. Here is a brief overview of each variable:

Variable	Description
gender	the individual's biological sex
age	the individual's age in years
hypertension	indicator for hypertension, a condition that causes increased blood pressure in arteries
heart_disease	indicator for heart disease
smoking_history	never, former, or current smoker
bmi	the individual's body mass index
HbA1c_level	Hemoglobin A1c level, a measure of the individual's average blood sugar level over the past 2-3 months
blood_glucose_level	the amount of glucose in the individuals bloodstream at a given time
diabetes	indicator for diabetes

'hypertension', 'heart_disease', and 'diabetes' are indicators, so they need to be converted to factors.

```r
diabetes_data$hypertension <- factor(diabetes_data$hypertension)
diabetes_data$heart_disease <- factor(diabetes_data$heart_disease)
diabetes_data$diabetes <- factor(diabetes_data$diabetes)
```

Check for missing data:

```r
sum(is.na(diabetes_data))
```

```
## [1] 0
```

Technically there are no missing values in the data, but notice that for the predictor 'smoking_history' there are some responses with 'No Info'. These should be counted as missing data and need to be dealt with.

```r
sum(diabetes_data$smoking_history == 'No Info')
```

```
## [1] 35816
```

There are 35,816 missing values for the predictor 'smoking_history'. Since there are 100,000 total observations, these values can simply be dropped with enough observations to spare.

```r
diabetes_data <- subset(diabetes_data, smoking_history != 'No Info')
dim(diabetes_data)
```

```
## [1] 64184     9
```

There are still 64,184 observations, which is more than enough to start creating machine learning models.

To begin, I split the data into training and testing set. I chose to go with the standard 70-30 split, which means that 70% of the data will be used for training and the other 30% will be used for testing and model evaluation.

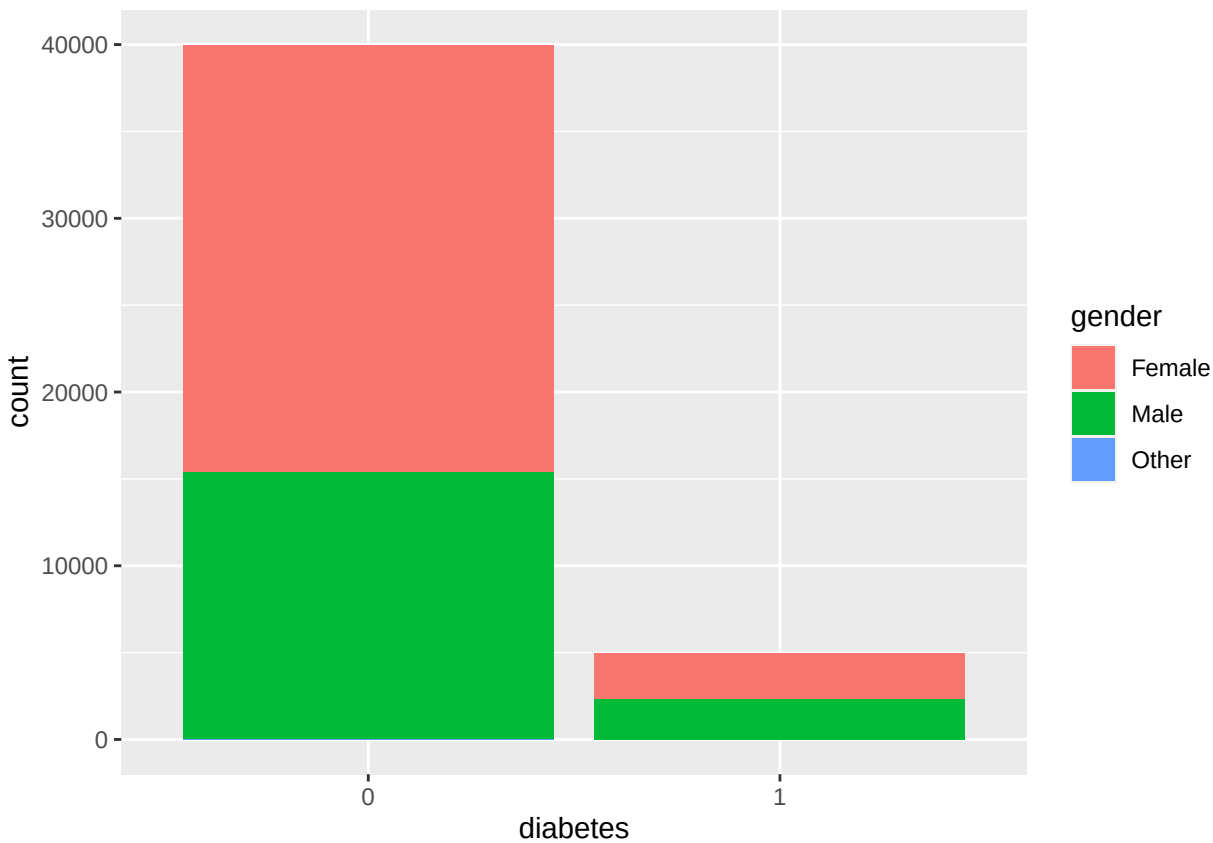
```
set.seed(321)
diabetes_split <- initial_split(diabetes_data, prop = 0.7, strata = diabetes)
diabetes_train <- training(diabetes_split)
diabetes_test <- testing(diabetes_split)
```

Here I created a k-fold validation set with k=5 folds. I chose to use k=5 instead of k=10 because my dataset is quite large, so using 10 folds would take large amounts of computing power and time. I used stratified sampling on the variable being predicted, which is 'diabetes', to ensure that there is an equal spread of each response in the validation set.

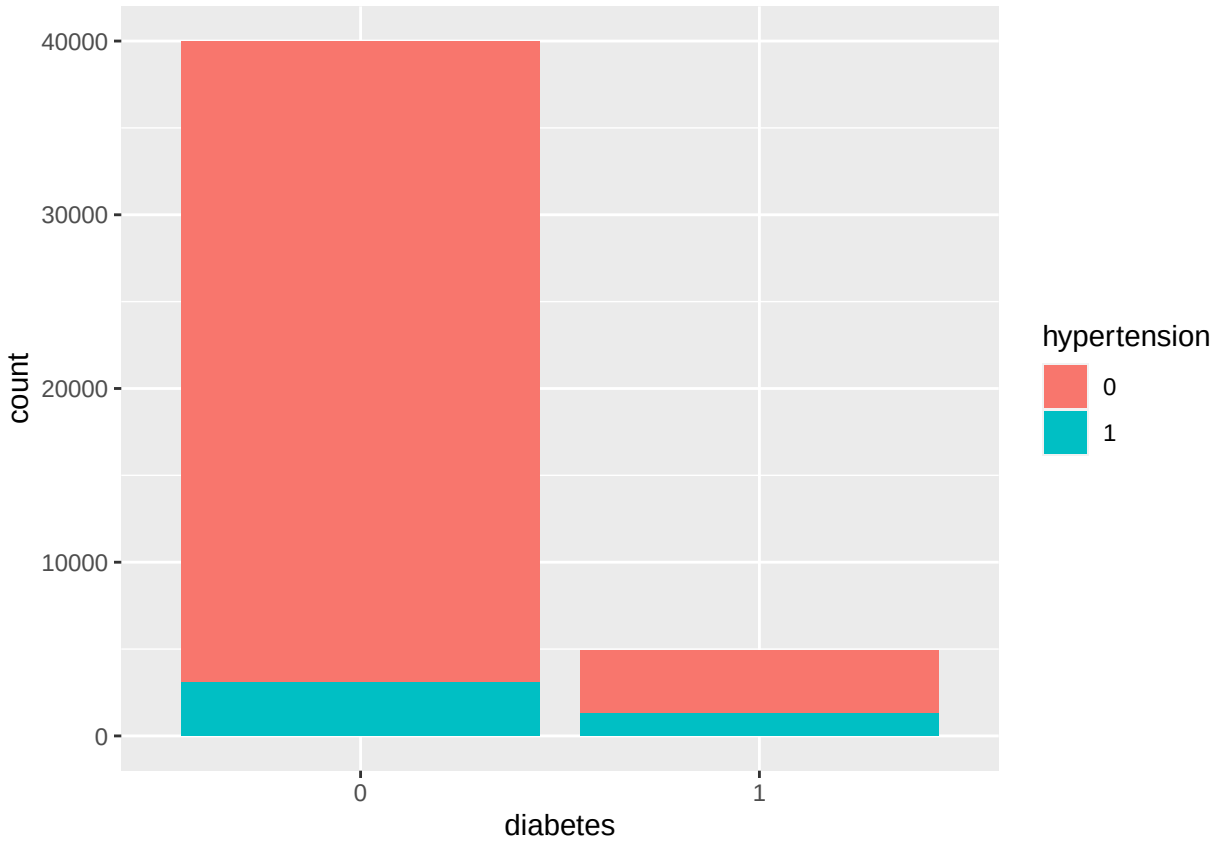
```
diabetes_folds <- vfold_cv(diabetes_train, v = 5, strata = diabetes)
```

Before fitting the data, let's examine the distribution of diabetes and how it is influenced by other categorical predictors.

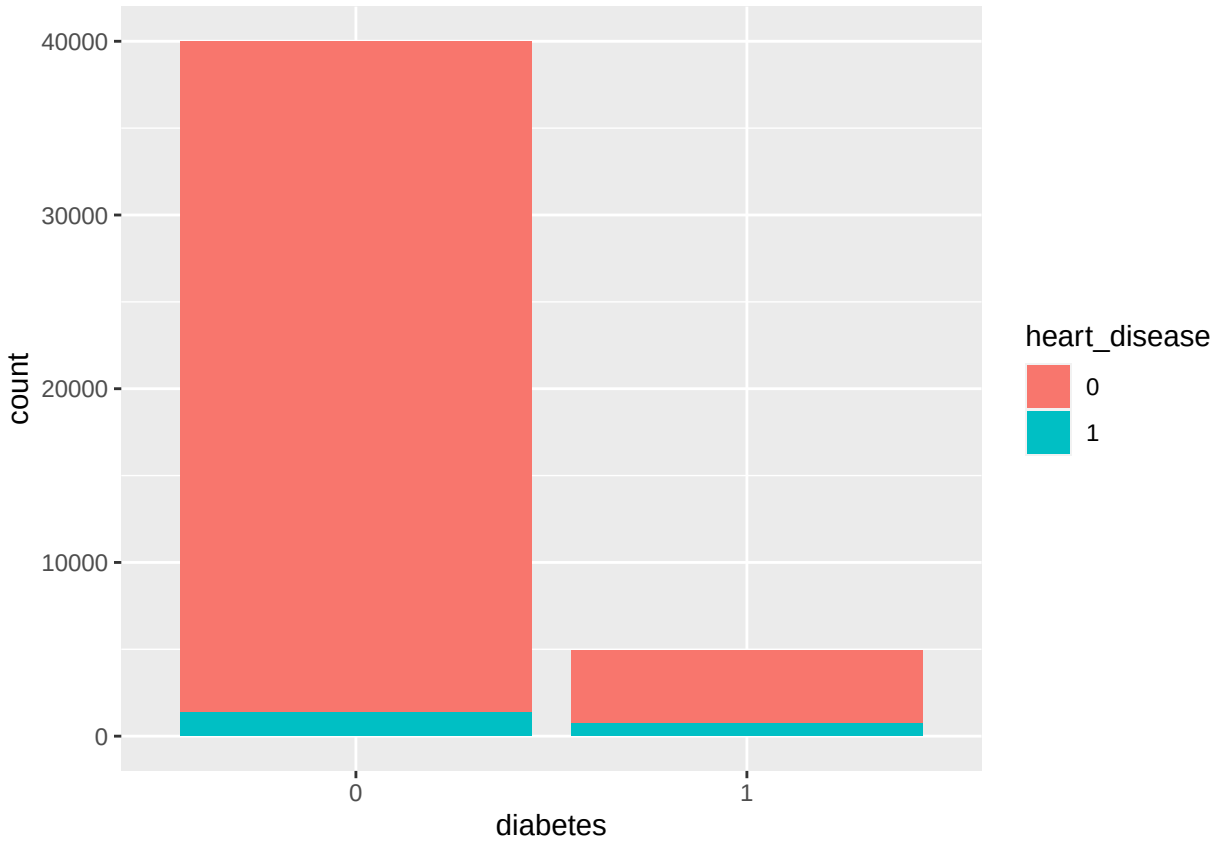
```
ggplot(diabetes_train) + geom_bar(aes(x = diabetes, fill = gender))
```



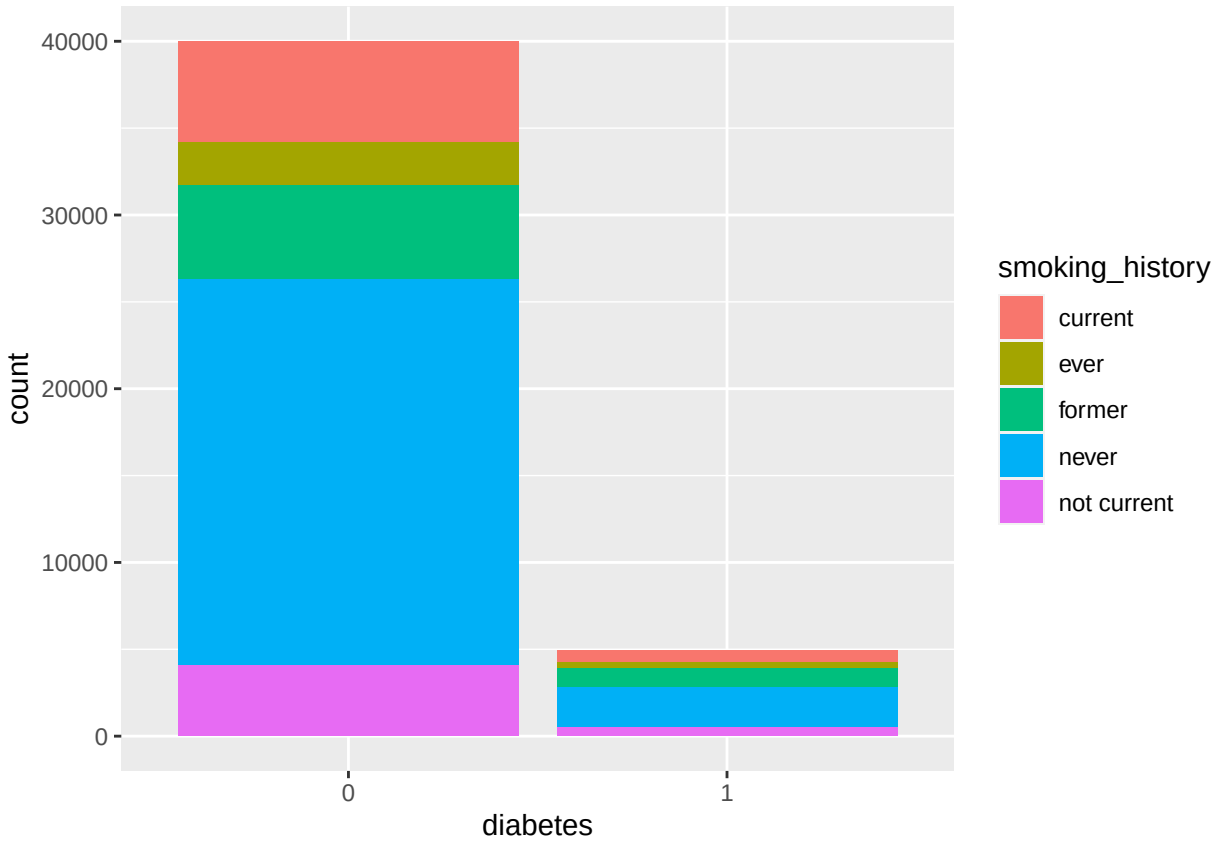
```r
ggplot(diabetes_train) + geom_bar(aes(x = diabetes, fill = hypertension))
```




```r
ggplot(diabetes_train) + geom_bar(aes(x = diabetes, fill = heart_disease))
```



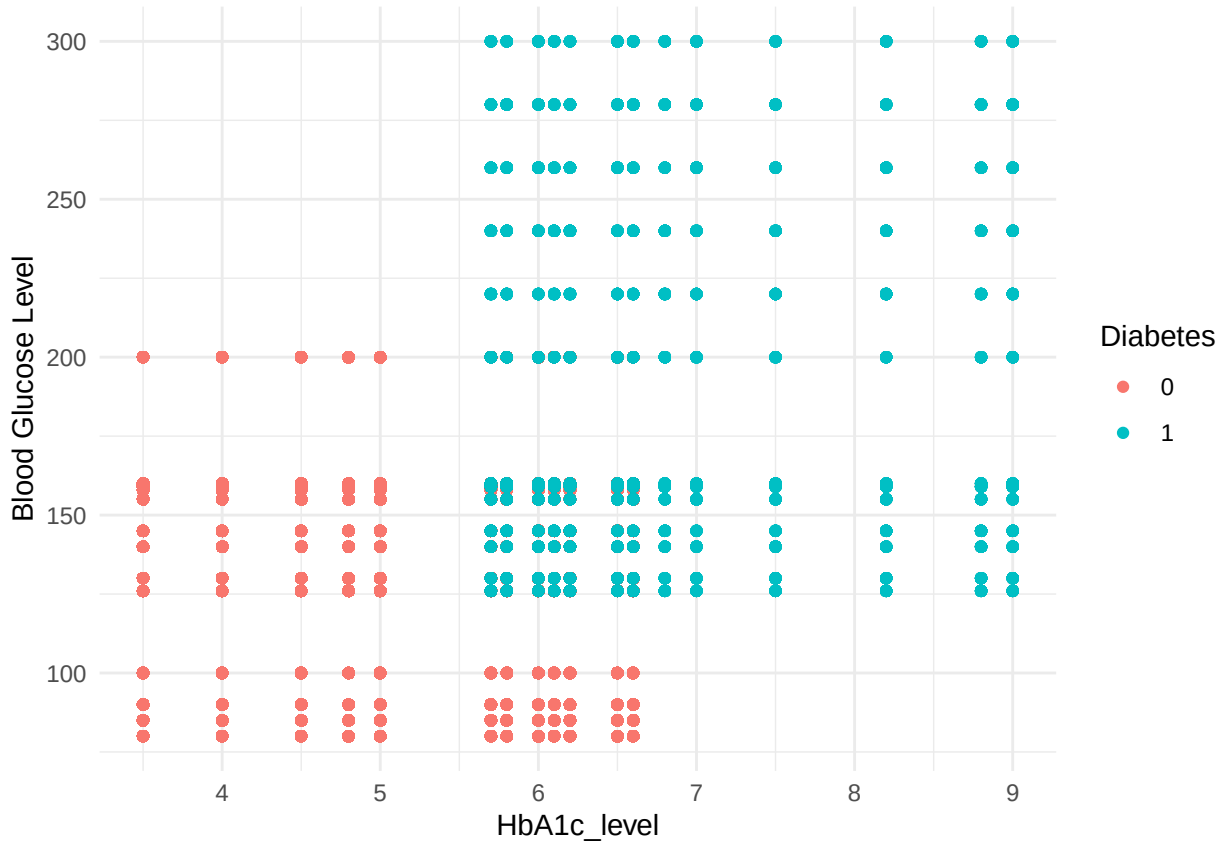

```
ggplot(diabetes_train) + geom_bar(aes(x = diabetes, fill = smoking_history))
```



It looks like 'hypertension' and 'heart_disease' may be good predictors of diabetes. This is because the relative proportion of people with these conditions is much higher when they have diabetes compared to not having diabetes. Notice that there are a lot more 0's than 1's in 'diabetes', so we will need to upsample the 1's when making the recipe.

Let's see how 'diabetes' is distributed among continuous predictors.

```
diabetes_train %>%
  ggplot(aes(x = HbA1c_level, y = blood_glucose_level, color = diabetes)) +
  geom_point() +
  theme_minimal() +
  xlab('HbA1c_level') +
  ylab('Blood Glucose Level') +
  labs(color = 'Diabetes')
```



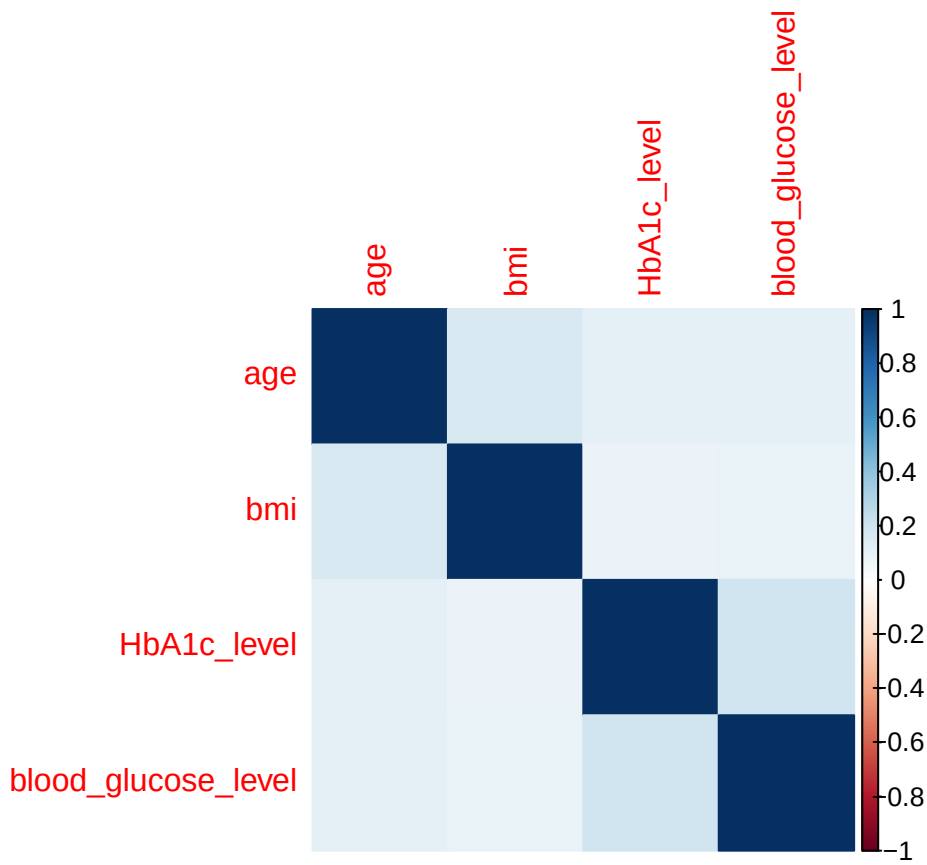
It appears that patients with high HbA1c levels, high blood glucose levels, or both are much more likely to have diabetes. If an individual has HbA1c level above 5.5 AND blood glucose level above 125, they have a high chance of also having diabetes.

Correlation matrix

The purpose of the correlation matrix is to see if any of the predictors are related to other predictors. If so, I may be able to combine any related predictors into a single broad predictor.

```
diabetes_train %>%
  dplyr::select(is.numeric) %>%
  cor() %>%
  corrplot(method = 'color')
```

```
## Warning: Use of bare predicate functions was deprecated in tidyselect 1.1.0.
## i Please use wrap predicates in `where()` instead.
##   # Was:
##   data %>% select(is.numeric)
##
##   # Now:
##   data %>% select(where(is.numeric))
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



There does not seem to be any significant correlation between any of the numerical predictors, which is good.

Training and Fitting Models

The first step in fitting the models is to create a recipe so that the models know which variable is being predicted and which variables to include as predictors. Since these models can only work with numeric predictors, any non-numeric predictors need to be dummy coded into numeric form. Another important step is to upsample 'diabetes' because its distribution is extremely uneven. Finally, I chose to normalize all the predictors just so that it is easier to work with some models.

```
diabetes_recipe <- recipe(diabetes ~ ., data = diabetes_train) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_upsample(diabetes, over_ratio = 1) %>%
  step_normalize(all_predictors())
```

Model Setup

Here I am essentially creating a skeleton for each of the models I plan to use. I need to specify which engine to use and whether I am doing regression or classification. For my data, I am choosing to do classification. Many of the models I use also use hyperparameters, which is a value I can set in order to control the complexity and depth of the model. However, right now I have no way of knowing what value is best to use for these hyperparameters. I do not want to simply choose the values that result in the greatest complexity because that could lead to potential overfitting. Instead, I set these values to tune(), which allows me to test out multiple values for each hyperparameter and pick the ones that result in the best fit.

```r
knn_mod <- nearest_neighbor(neighbors = tune()) %>%
  set_engine('kknn') %>%
  set_mode('classification')

log_reg <- logistic_reg() %>%
  set_engine('glm') %>%
  set_mode('classification')

lda_mod <- discrim_linear() %>%
  set_engine('MASS') %>%
  set_mode('classification')

qda_mod <- discrim_quad() %>%
  set_engine('MASS') %>%
  set_mode('classification')

en_mod <- logistic_reg(mixture = tune(), penalty = tune()) %>%
  set_engine('glmnet') %>%
  set_mode('classification')

rf_mod <- rand_forest(mtry = tune(), trees = tune(), min_n = tune()) %>%
  set_engine('ranger') %>%
  set_mode('classification')

bt_mod <- boost_tree(mtry = tune(), trees = tune(), learn_rate = tune()) %>%
  set_engine('xgboost') %>%
  set_mode('classification')

svm_lin_mod <- svm_poly(degree = 1, cost = tune()) %>%
  set_engine('kernlab') %>%
  set_mode('classification')

svm_rbf_mod <- svm_rbf(cost = tune()) %>%
  set_engine('kernlab') %>%
  set_mode('classification')
```

Create workflows

Next I simply add each model and my recipe to a separate workflow so that I can begin fitting each model.

```r
knn_wflow <- workflow() %>%
  add_model(knn_mod) %>%
  add_recipe(diabetes_recipe)

log_wflow <- workflow() %>%
  add_model(log_reg) %>%
  add_recipe(diabetes_recipe)

lda_wflow <- workflow() %>%
  add_model(lda_mod) %>%
  add_recipe(diabetes_recipe)
```

```r
qda_wflow <- workflow() %>%
  add_model(qda_mod) %>%
  add_recipe(diabetes_recipe)

en_wflow <- workflow() %>%
  add_model(en_mod) %>%
  add_recipe(diabetes_recipe)

rf_wflow <- workflow() %>%
  add_model(rf_mod) %>%
  add_recipe(diabetes_recipe)

bt_wflow <- workflow() %>%
  add_model(bt_mod) %>%
  add_recipe(diabetes_recipe)

svm_lin_wflow <- workflow() %>%
  add_model(svm_lin_mod) %>%
  add_recipe(diabetes_recipe)

svm_rbf_wflow <- workflow() %>%
  add_model(svm_rbf_mod) %>%
  add_recipe(diabetes_recipe)
```

Create grids

In order to tune the hyperparameters, I first need to create a grid with a specified range of values for each hyperparameter. It would be nice to create a huge grid with a wide range and many levels, and I would be sure to find the best possible values. However, I need to take into account the amount of time it would take to compute every possible combination of values, which could be a very long time. Hopefully the ranges I specified strikes a good balance between efficiency and effectiveness.

```r
knn_grid <- grid_regular(neighbors(range = c(1,10)),
                         levels = 10)

en_grid <- grid_regular(penalty(range = c(0,1),
                        trans = identity_trans()),
                        mixture(range = c(0,1)),
                        levels = 10)

rf_grid <- grid_regular(mtry(range = c(1, 8)),
                        trees(range = c(200, 600)),
                        min_n(range = c(10, 20)),
                        levels = 8)

bt_grid <- grid_regular(mtry(range = c(1, 8)),
                        trees(range = c(200, 600)),
                        learn_rate(range = c(-10, -1)),
                        levels = 8)

svm_lin_grid <- grid_regular(cost(),
                             levels = 5)
```

```r
svm_rbf_grid <- grid_regular(cost(),
                             levels = 5)
```

Tune parameters

Now we want to find out which combination of hyperparameters result in the highest model performance. The only way to know for sure is to test out each combination individually, so a lot of models are going to be fit in this process. Along with the fact that the dataset includes tens of thousands of observations, this step is going to take a long time.

```r
knn_tune_res <- tune_grid(
  knn_wflow,
  resamples = diabetes_folds,
  grid = knn_grid,
  control = control_grid(verbose = TRUE)
)
```

```r
en_tune_res <- tune_grid(
  en_wflow,
  resamples = diabetes_folds,
  grid = en_grid,
  control = control_grid(verbose = TRUE)
)
```

```r
rf_tune_res <- tune_grid(
  rf_wflow,
  resamples = diabetes_folds,
  grid = rf_grid,
  control = control_grid(verbose = TRUE)
)
```

```r
bt_tune_res <- tune_grid(
  bt_wflow,
  resamples = diabetes_folds,
  grid = bt_grid,
  control = control_grid(verbose = TRUE)
)
```

```r
svm_lin_tune_res <- tune_grid(
  svm_lin_wflow,
  resamples = diabetes_folds,
  grid = svm_lin_grid,
  control = control_grid(verbose = TRUE)
)
```

```r
svm_rbf_tune_res <- tune_grid(
  svm_rbf_wflow,
  resamples = diabetes_folds,
  grid = svm_rbf_grid,
  control = control_grid(verbose = TRUE)
)
```

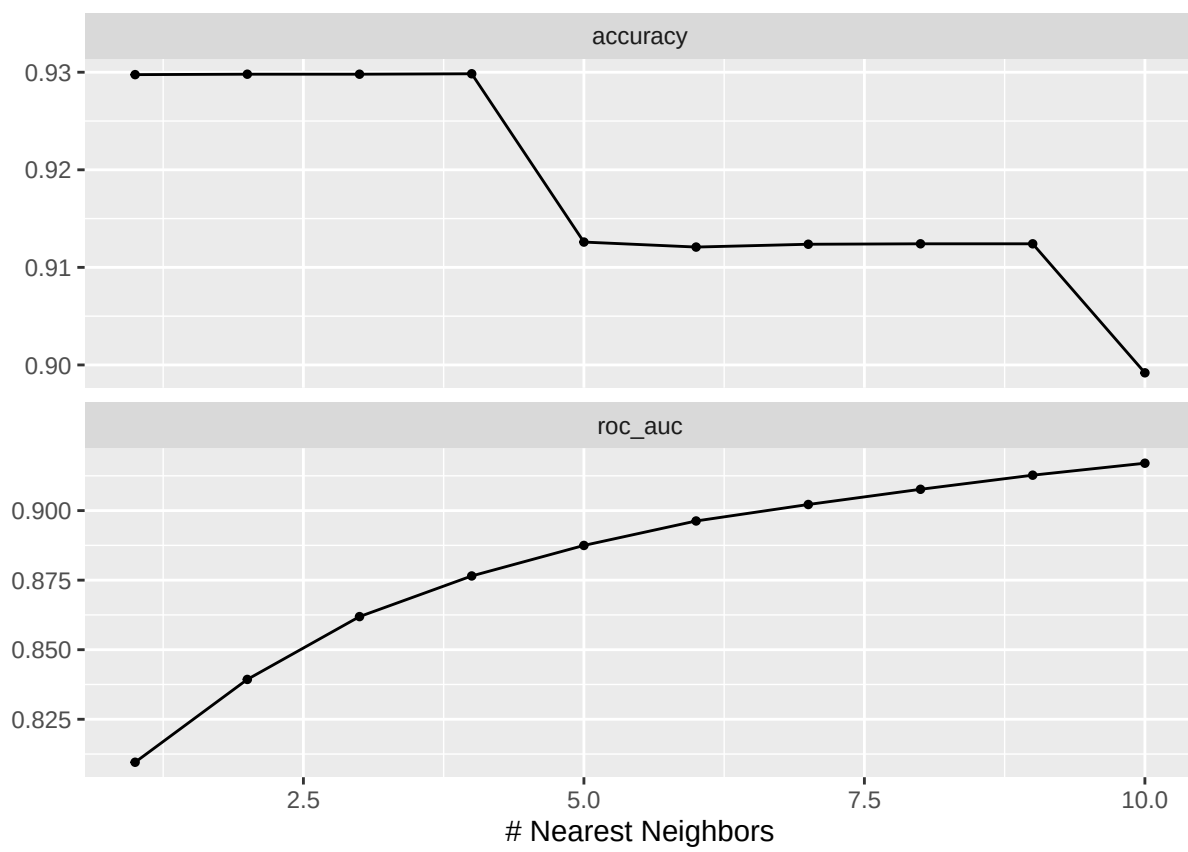
Some of these models took a lot of computational power and time. The random forest took the longest with 5 hours, which is around 1 hour for each individual fold. The rest of the models took around an hour each to tune parameters.

```
save(knn_tune_res, file = 'knn_tune_res.rda')
save(en_tune_res, file = 'en_tune_res.rda')
save(rf_tune_res, file = 'rf_tune_res.rda')
save(bt_tune_res, file = 'bt_tune_res.rda')
save(svm_lin_tune_res, file = 'svm_lin_tune_res.rda')
save(svm_rbf_tune_res, file = 'svm_rbf_tune_res.rda')
```

```
load('knn_tune_res.rda')
load('en_tune_res.rda')
load('rf_tune_res.rda')
load('bt_tune_res.rda')
load('svm_lin_tune_res.rda')
load('svm_rbf_tune_res.rda')
```

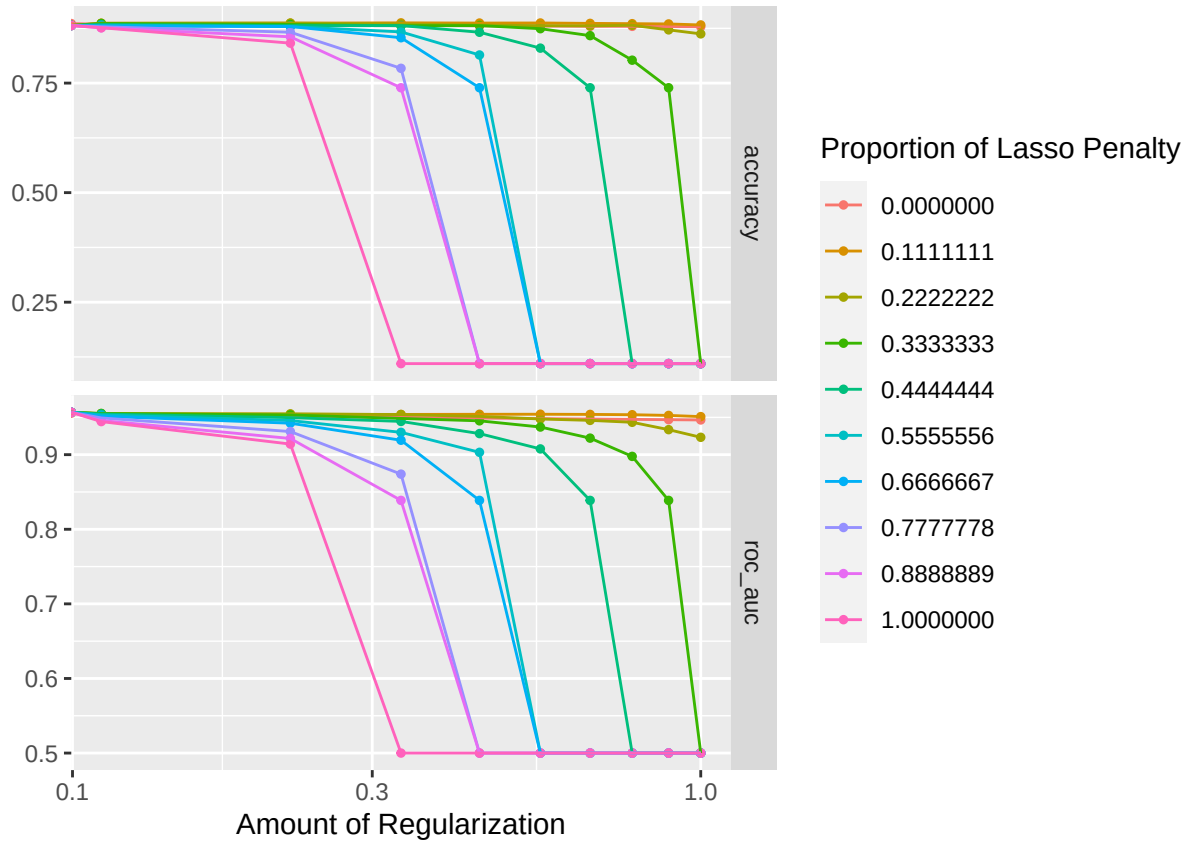
We can visualize the results of the hyperparameter tuning too see which values performed the best. The main metric we are interested is area under the ROC curve.

```
autoplot(knn_tune_res)
```

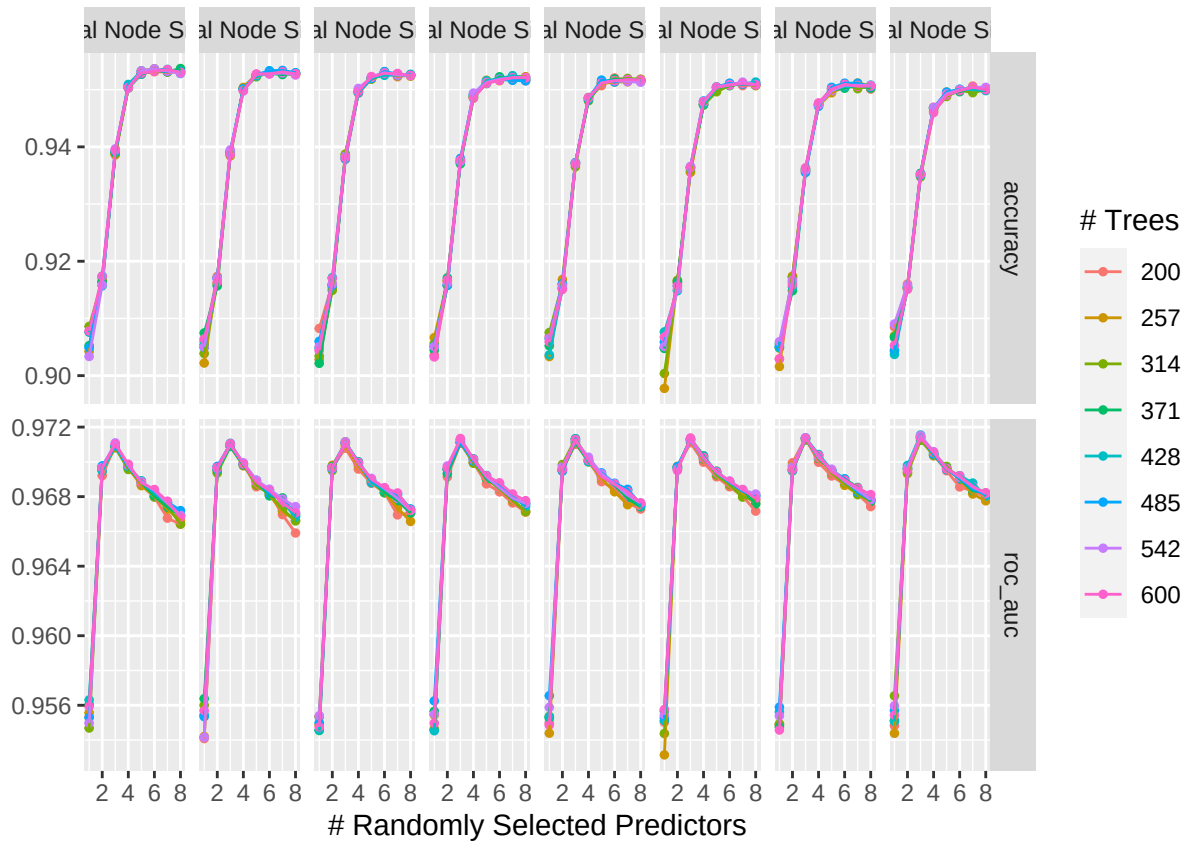


```r
autoplot(en_tune_res)
```

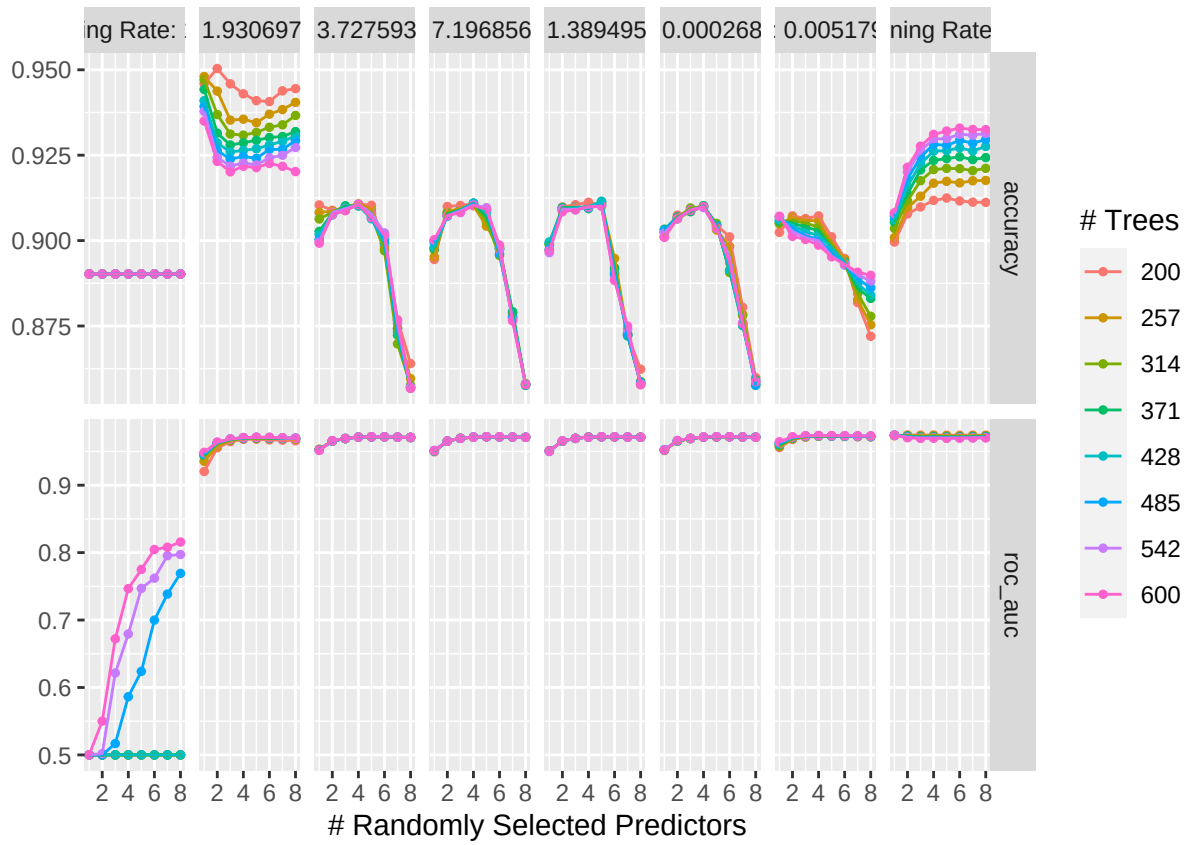
```
## Warning: Transformation introduced infinite values in continuous x-axis
## Transformation introduced infinite values in continuous x-axis
```



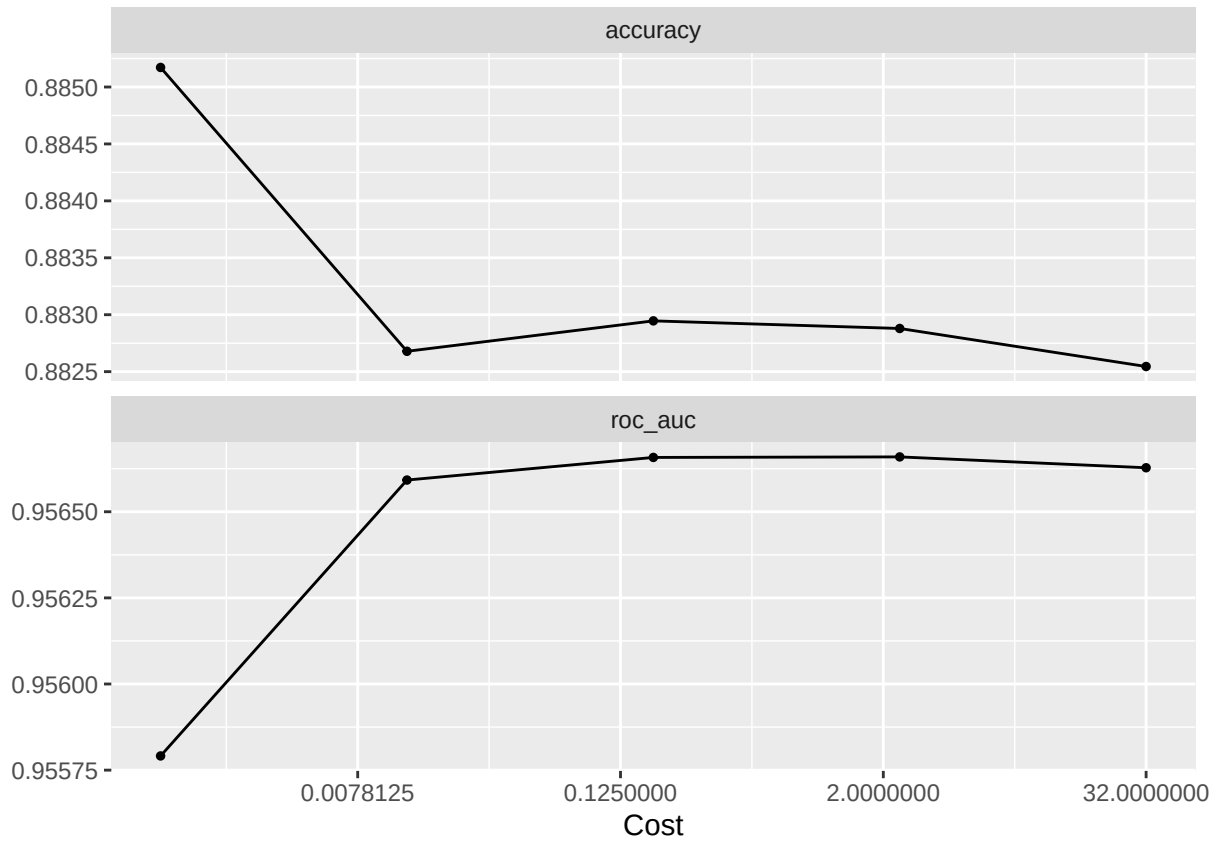

```r
autoplot(rf_tune_res)
```



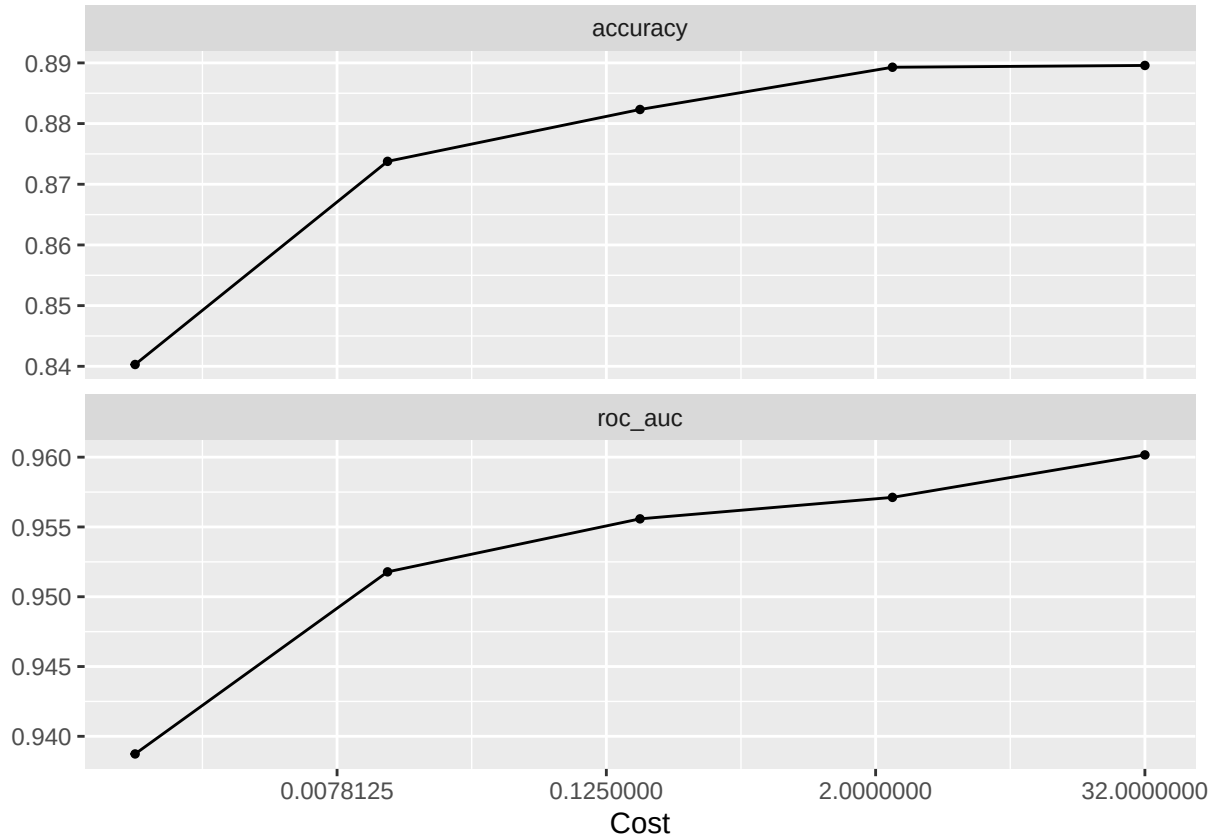

```
autoplot(bt_tune_res)
```




```
autoplot(svm_lin_tune_res)
```




```r
autoplot(svm_rbf_tune_res)
```



For some models, there is a clear answer as to which hyperparamter values performed the best. For example, in KNN and both SVM models, higher values tend to result in high ROC AUC. In random forest, it is hard to tell which amount of trees results in the best prediction accuracy because they all perform similarly. The same is true for the boosted trees model. Luckily, there are functions in R that can determine which values are truly the best.

Fit models without tuned parameters

The logistic regression, LDA, and QDA models do not require hyperparameter tuning, so they can be fit directly and quickly.

```
log_fit <- log_wflow %>%
  fit_resamples(diabetes_folds)

lda_fit <- lda_wflow %>%
  fit_resamples(diabetes_folds)
```

I was not able to fit the QDA model because of a rank deficiency, so that model will not be included.

Compare area under ROC curve

Now that we have fit all the models, we can compare each one's prediction accuracy. To measure prediction accuracy, we use a metric called area under ROC curve, which has values that range from 0 to 1. Higher values indicate better performance, so we want to select the model that has an area closest to 1.

The following code displays the values we should tune the hyperparameters that maximize performance along with the corresponding area under the ROC curve.

```
show_best(knn_tune_res, metric = 'roc_auc', n = 1)
```

```
## # A tibble: 1 x 7
##   neighbors .metric .estimator  mean     n std_err .config
##       <int> <chr>   <chr>      <dbl> <int>   <dbl> <chr>
## 1        10 roc_auc binary     0.917     5 0.00389 Preprocessor1_Model10
```

```
show_best(en_tune_res, metric = 'roc_auc', n = 1)
```

```
## # A tibble: 1 x 8
##   penalty mixture .metric .estimator  mean     n std_err .config
##     <dbl>   <dbl> <chr>   <chr>      <dbl> <int>   <dbl> <chr>
## 1       0   0.111 roc_auc binary     0.957     5 0.00185 Preprocessor1_Model011
```

```
show_best(rf_tune_res, metric = 'roc_auc', n = 1)
```

```
## # A tibble: 1 x 9
##    mtry trees min_n .metric .estimator  mean     n std_err .config
##   <int> <int> <int> <chr>   <chr>      <dbl> <int>   <dbl> <chr>
## 1     3   428    20 roc_auc binary     0.972     5 0.00151 Preprocessor1_Model4~
```

```
show_best(bt_tune_res, metric = 'roc_auc', n = 1)
```

```
## # A tibble: 1 x 9
##    mtry trees learn_rate .metric .estimator  mean     n std_err .config
##   <int> <int>      <dbl> <chr>   <chr>      <dbl> <int>   <dbl> <chr>
## 1     4   200        0.1 roc_auc binary     0.974     5 0.00103 Preprocessor1_M~
```

```
show_best(svm_lin_tune_res, metric = 'roc_auc', n = 1)
```

```
## # A tibble: 1 x 7
##    cost .metric .estimator  mean     n std_err .config
##   <dbl> <chr>   <chr>      <dbl> <int>   <dbl> <chr>
## 1  2.38 roc_auc binary     0.957     5 0.00189 Preprocessor1_Model4
```

```
show_best(svm_rbf_tune_res, metric = 'roc_auc', n = 1)
```

```
## # A tibble: 1 x 7
##    cost .metric .estimator  mean     n std_err .config
##   <dbl> <chr>   <chr>      <dbl> <int>   <dbl> <chr>
## 1    32 roc_auc binary     0.960     5 0.00178 Preprocessor1_Model5
```

```
collect_metrics(log_fit)
```

```
## # A tibble: 2 x 6
##   .metric  .estimator  mean     n std_err .config
##   <chr>    <chr>      <dbl> <int>   <dbl> <chr>
## 1 accuracy binary     0.881     5 0.00125 Preprocessor1_Model1
## 2 roc_auc  binary     0.957     5 0.00185 Preprocessor1_Model1
```

```r
collect_metrics(lda_fit)
```

```
## # A tibble: 2 x 6
##   .metric  .estimator  mean     n  std_err .config
##   <chr>    <chr>      <dbl> <int>    <dbl> <chr>
## 1 accuracy binary     0.892     5 0.000743 Preprocessor1_Model1
## 2 roc_auc  binary     0.955     5 0.00204  Preprocessor1_Model1
```

Model Selection

Model	Area under ROC curve
Logistic	0.9568
LDA	0.9555
KNN	0.9170
Elastic net	0.9568
Random forest	0.9715
Boosted trees	0.9744
SVM linear	0.9567
SVM radial	0.9602

All models performed extremely well, but the best performing model was boosted trees with an ROC AUC of 0.9744. The specific hyperparameter values used in this model are mtry=4, trees=200, and learn_rate=0.1.The next step is to create a finalized workflow using the boosted trees model with the tuned hyperparamter values. This is the model we are going to train on the training set and test on the testing set.

```r
best_bt_param <- show_best(bt_tune_res, metric = 'roc_auc', n = 1)

final_diabetes_wflow <- finalize_workflow(bt_wflow, best_bt_param)
final_diabetes_fit <- fit(final_diabetes_wflow, diabetes_train)
```

Assessing Performance

The last step is to see how this model performs on the testing data. We have a good estimate of performance based on the k-fold validation set, but this does not necessarily mean it will also perform well on the testing set.

```r
augment(final_diabetes_fit, new_data = diabetes_test) %>%
  roc_auc(diabetes, .pred_0)
```

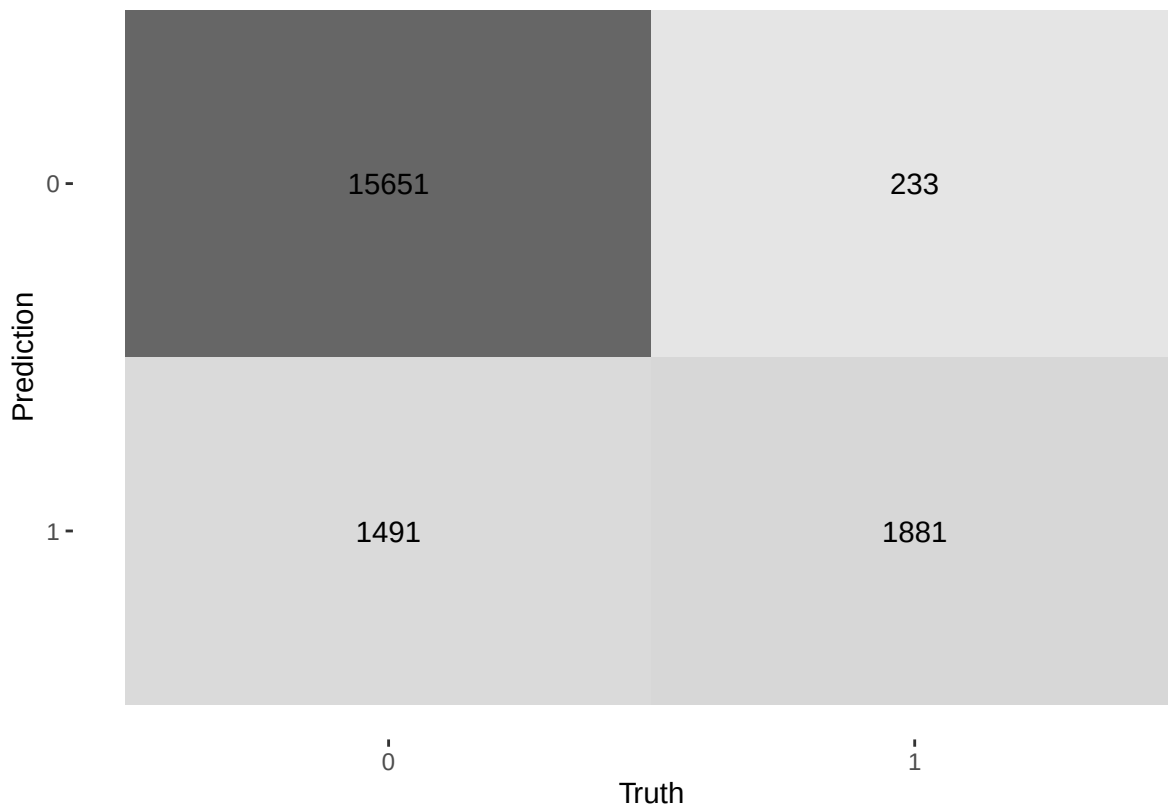
```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 roc_auc binary         0.975
```

Surprisingly, the model's testing ROC AUC of 0.9746 is actually just slightly higher than the average ROC AUC across folds of 0.9744. Therefore, this model performed extremely well on the testing data and can likely be used to effectively predict diabetes.

Confusion Matrix

We can view a confusion matrix that gives a breakdown of the model's performance on the testing data and where it made mistakes in predicting.

```
augment(final_diabetes_fit, new_data = diabetes_test) %>%
  conf_mat(diabetes, .pred_class) %>%
  autoplot('heatmap')
```

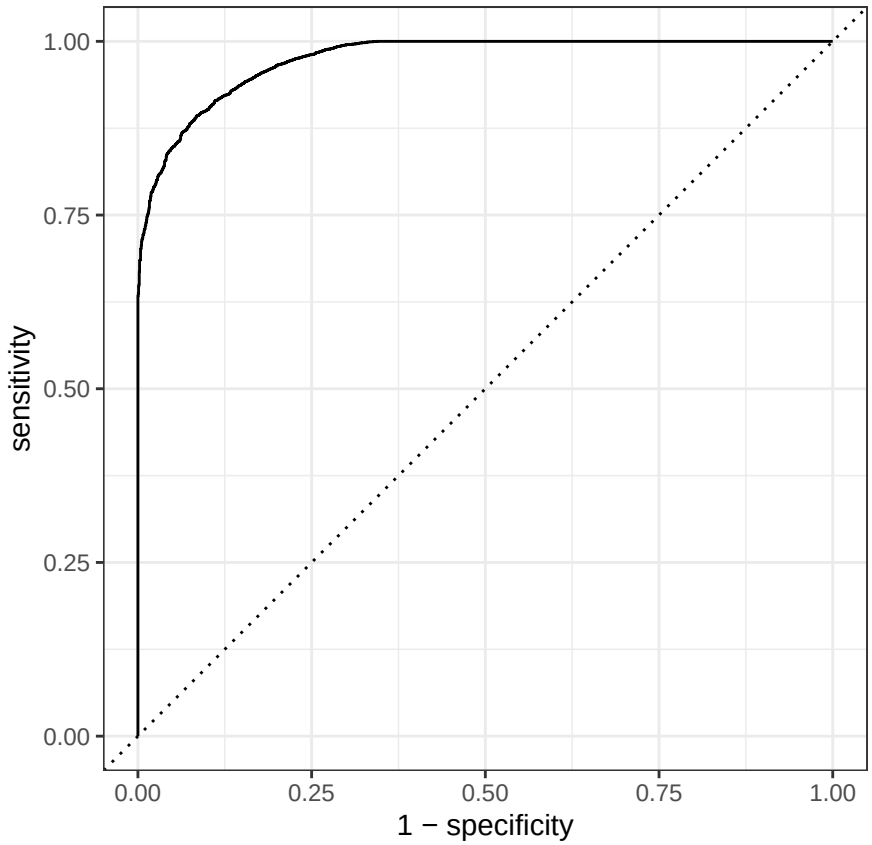


A large majority of the predictions were true negatives, where the model correctly predicted that an individual did not have diabetes. We may be more interested in cases where the patient did have diabetes. In that case, the model correctly predicted 1874 times but was unable to come to that conclusion 274 times. There were also 1478 false positives, which is when it predicts that a person has diabetes when in fact they do not.

ROC curve

Another way to visualize these results is with an ROC curve.

```
augment(final_diabetes_fit, new_data = diabetes_test) %>%
  roc_curve(diabetes, .pred_0) %>%
  autoplot()
```



Conclusion

Now we have a model that can predict whether or not a person has diabetes to a high degree. For this particular data, using a boosted tree model resulted in the greates success, although all the other models also worked well. A much simpler model, such as a logistic regression, was able to perform nearly as well as our final model without having to deal with the time consuming process of tuning parameters. Because of the large number of observations and the fact that this is a binary classificatiob, it makes sense that any model can be trained to perform well. I was a bit let down my the random forest model. Although it had a great area under ROC curve, it took 5 hours to fit the model and it was still outclassed by the boosted trees model.

Further steps can be taken to improve this model. For instance, we can do a principal component analysis to determine which predictors are most useful in predicting the outcome. This can give us a simpler, easier to use model that still makes accurate predictions.

This model can be used by medical professionials to see whether patients are at risk of having diabetes. It can take people months or even years before they realize that they have diabetes, which can lead to complications later on. If diabetes can be predicted near its onset, steps can be taken to treat it and mitigate its effect.